Bài toán tìm kiếm

- Tìm kiếm cái gì?
- Biểu diễn và tìm kiếm là kỹ thuật phổ biến giải các bài toán trong lĩnh vực AI
- Các vấn đề khó khăn trong tìm kiếm với các bài toán AI
  - Đặc tả vấn đề phức tạp
  - Không gian tìm kiếm lớn
  - Đặc tính đối tượng tìm kiếm thay đổi
  - Đáp ứng thời gian thực
  - Meta knowledge và kết quả "tối ưu"
- Khó khăn về kỹ thuật

Cấu trúc chung của bài toán tìm kiếm

- Một cách chung nhất, nhiều vấn đề-bài toán phức tạp đều có dạng "tìm đường đi trong đồ thị" hay nói một cách hình thức hơn là *"xuất phát từ một đỉnh của một đồ thị, tìm đường đi hiệu quả nhất đến một đỉnh nào đó"*.
- Một phát biểu khác thường gặp của dạng bài toán này là:

  Cho trước hai trạng thái T_0 và T_G hãy xây dựng chuỗi trạng thái $T_0, T_1, T_2, ..., T_{n-1}, T_n = T_G$ sao cho :

$$\sum_1^n cost(T_{i-1}, T_i)$$

  thỏa mãn một điều kiện cho trước (thường là nhỏ nhất).

2.2 Giải thuật tổng quát

- Ký hiệu:

  s đỉnh xuất phát

  g: đỉnh đích

  n: đỉnh đang xét

  Γ(n): tập các đỉnh có thể đi trực tiếp từ đỉnh n

  Open: tập các đỉnh có thể xét ở bước kế tiếp

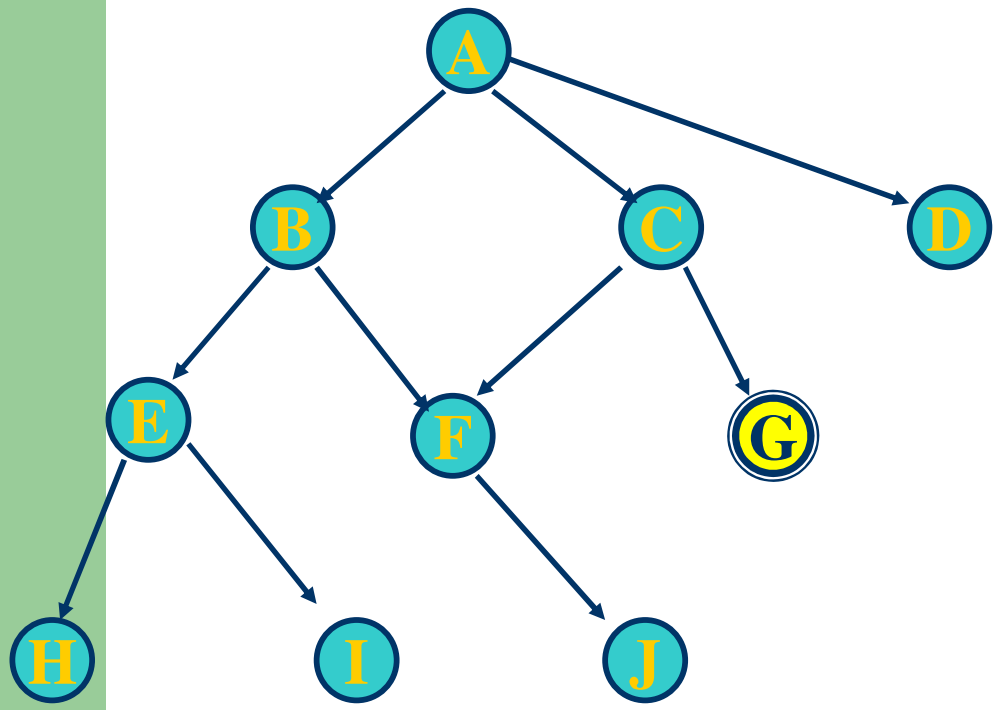
  Close: tập các đỉnh đã xét

2.2 Giải thuật tổng quát (tiếp)

```
Begin
    Open := {s};
    Close := ø;
    While (Open <> ø) do
        begin
            n:=Retrieve(Open);
            if (n=g) then Return True;
            Open := Open ∪ Γ(n); // (Γ(n) – Close)
            Close := Close ∪ {n};
        end;
    Return False;
End;
```

Ví dụ:

- Xét graph sau:

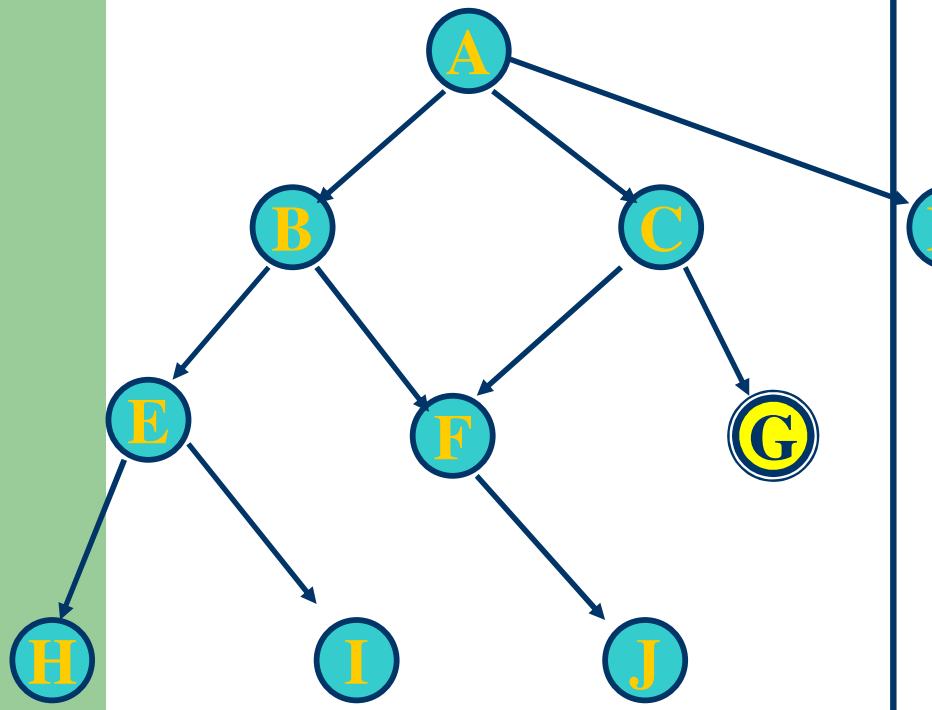


s = A là đỉnh bắt đầu

g= G là đỉnh đích

2.3 Breath First Search – Ví dụ

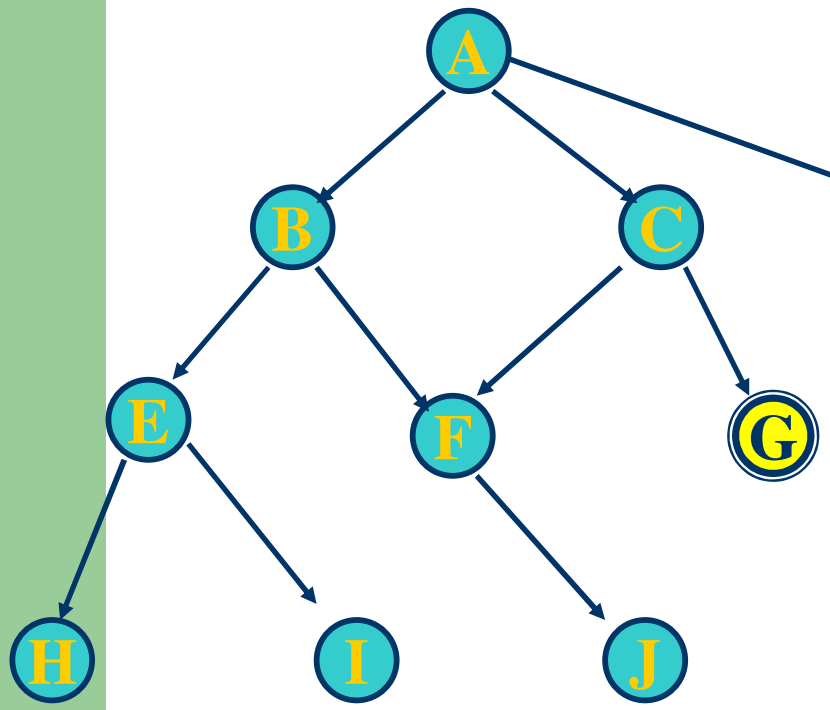
- Xét graph sau:



Lần lặp	n	Γ(n)	Open	Close
0			{A}	ø
1	A	{B, C, D}	{B, C, D}	{A}
2	B	{E, F}	{C, D, E, F}	{A, B}
3	C	{F, G}	{D, E, F, G}	{A, B, C}
4	D	ø	{E, F, G}	{A, B, C, D}
5	E	{H, I}	{F, G, H, I}	{A, B, C, D, E}
6	F	{J}	{G, H, I, J}	{A, B, C, D, E, F}
7	G			
		True		

2.3 Breath First Search – Ví dụ 1

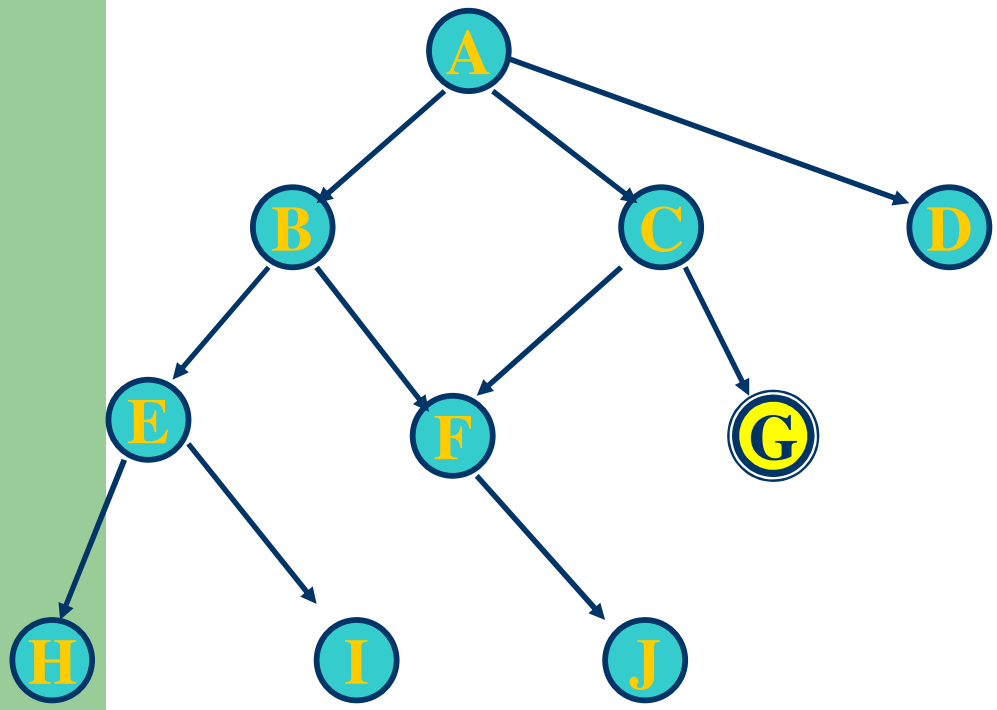
- Xét graph sau:A->U



Lần lặp	n	Γ(n)	Open	Close
0			{A}	ø
1	A	{B, C, D}	{B,C,D}	{A}
2	B	{E, F}	{C,D, E,F}	{A, B}
3	C	{F, G}	{D,E, F,G}	{A, B, C}
4	D	ø	{E, F, G}	{A, B, C, D}
5	E	{H, I}	{F, G, H, I}	{A, B, C, D, E}
6	F	{J}	{G, H, I, J}	{A, B, C, D, E, F}
7	G	ø	{H, I, J}	{A, B, C, D, E, F,G}
8	H	Ø	{I, J}	{A,B,C, D, E, F,G,H}
9	I	ø	{J}	{A,B,C, D, E, F,G,H,I}
10	J	Ø	Ø	{A,B,C, D, E, F,G,H,I,J}
		FALSE		

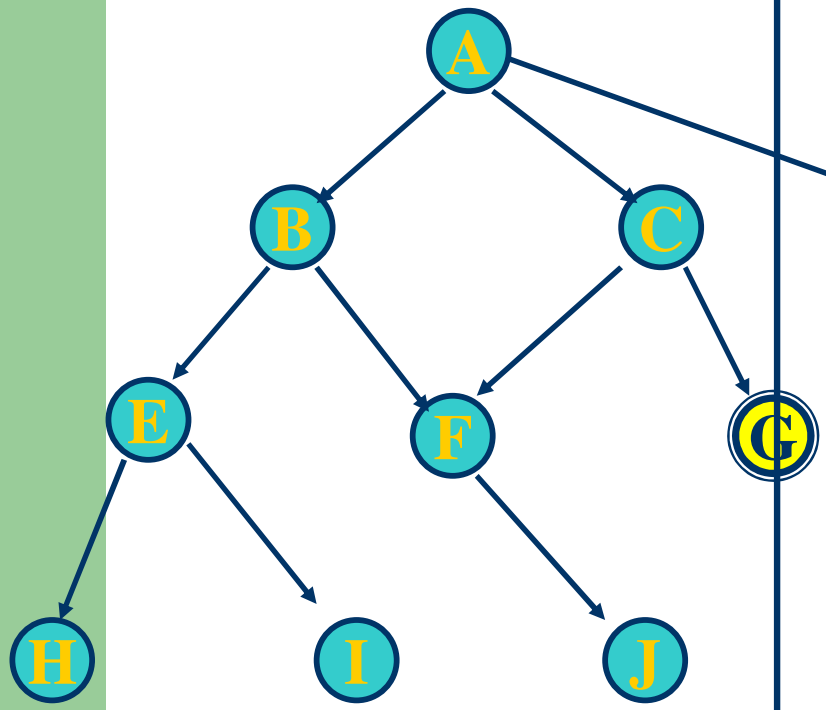
Ví dụ:

- Xét graph sau:



2.4 Depth First Search – Ví dụ

- Xét graph sau:



Lần lặp	n	Γ(n)	Open	Close
0			{A}	ø
1	A	{B, C, D}	{B, C, D}	{A}
2	B	{E, F}	{E, F, C, D}	{A, B}
3	E	{H, I}	{H, I, F, C, D}	{A, B, E}
4	H	Ø	{I, F, C, D}	{A, B, E, H}
5	I	Ø	{F, C, D}	{A, B, E, H, I}
6	F	{J}	{J, C, D}	{A, B, E, H, I, F}
7	J	Ø	{C, D}	{A, B, E, H, I, F,J}
8	C	{F, G}	{G, D}	{A,B,E,H,I, F,J,C}
9	G	True		

Breath First vs Depth First

- Breath First: open được tổ chức dạng FIFO
- Depth First: open được tổ chức dạng LIFO
- Hiệu quả
  - Breath First luôn tìm ra nghiệm có số cung nhỏ nhất
  - Depth First "thường" cho kết quả nhanh hơn.
- Kết quả
  - BFS, DFS chắc chắn tìm ra kết quả nếu có.
- Bùng nổ tổ hợp là khó khăn lớn nhất cho các giải thuật này.

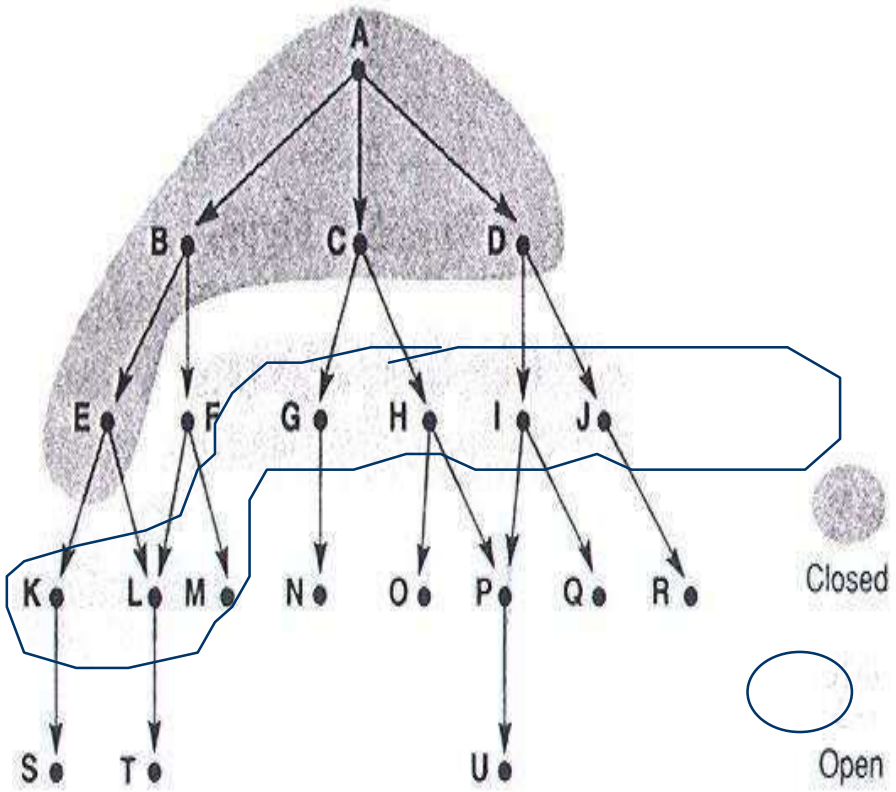
Giải Pháp cho bùng nổ tổ hợp??

Tìm Kiếm Rộng

1. Open = [A]; closed = []
2. Open = [B,C,D];
   closed = [A]
2. Open = [C,D,E,F];
   closed = [B,A]
3. Open = [D,E,F,G,H]; closed = [C,B,A]
4. Open = [E,F,G,H,I,J]; closed = [D,C,B,A]
5. Open = [F,G,H,I,J,K,L];closed = [E,D,C,B,A]
6. Open = [G,H,I,J,K,L,M];(vì L đã có trong open);

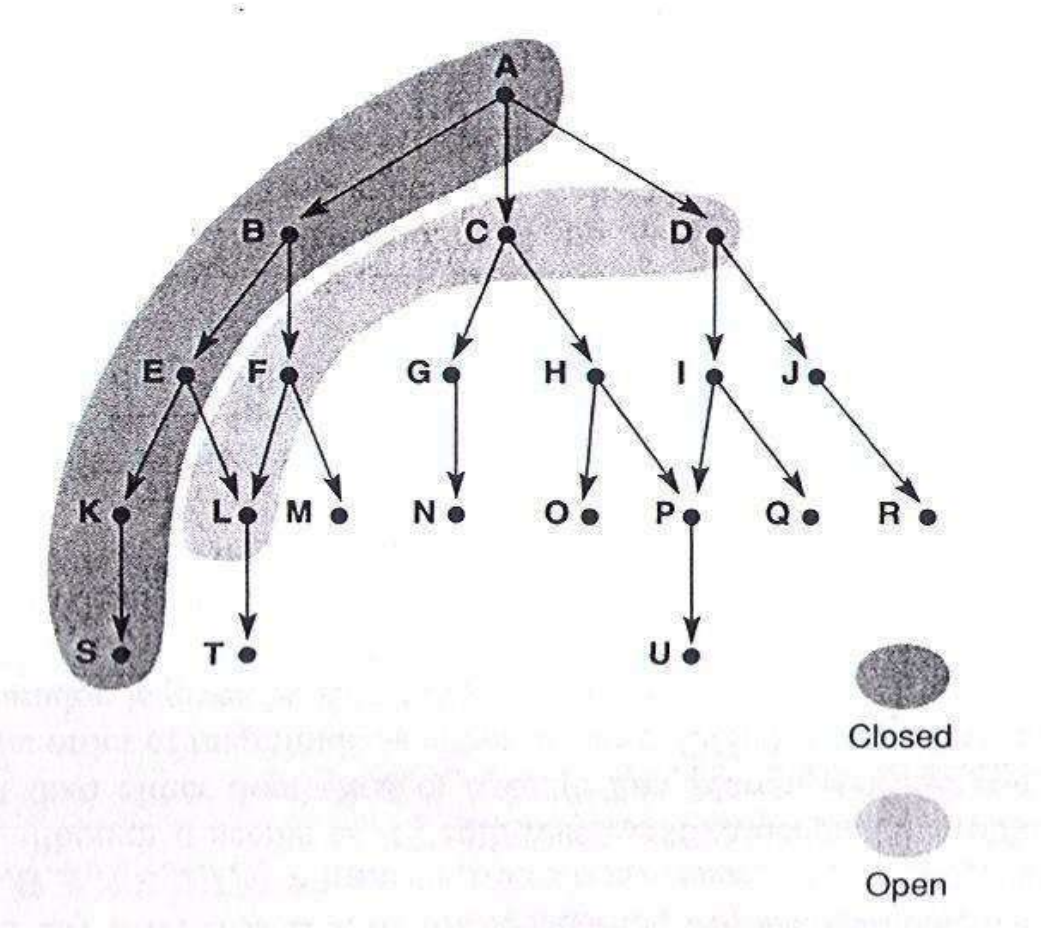
closed = [F,E,D,C,B,A]

…



Tìm kiếm Sâu

1. Open = [A]; closed = []
2. Open = [B,C,D]; closed = [A]
3. Open = [E,F,C,D];closed = [B,A]
4. Open = [K,L,F,C,D];
   closed = [E,B,A]
5. Open = [S,L,F,C,D];
   closed = [K,E,B,A]
6. Open = [L,F,C,D];
   closed = [S,K,E,B,A]
7. Open = [T,F,C,D];
   closed = [L,S,K,E,B,A]
8. Open = [F,C,D];
   closed = [T,L,S,K,E,B,A]
9. …



Depth first search có giới hạn

- Depth first search có khả năng lặp vô tận do các trạng thái con sinh ra liên tục. Độ sâu tăng vô tận.
- Khắc phục bằng cách giới hạn độ sâu của giải thuật.
- Sâu bao nhiêu thì vừa?
- Chiến lược giới hạn:
  - Cố định một độ sâu MAX, như các danh thủ chơi cờ tính trước được số nước nhất định
  - Theo cấu hình resource của máy tính
  - Meta knowledge trong việc định giới hạn độ sâu.
- Giới hạn độ sâu => co hẹp không gian trạng thái => có thể mất nghiệm.

3.1 Giới thiệu về Heuristic

- **Heuristic là gì?**
  - Heuristic là những tri thức được rút tỉa từ những kinh nghiệm, "trực giác" của con người.
  - Heuristic có thể là những tri thức "đúng" hay "sai".
  - Heuristic là những meta knowledge và "thường đúng".
- **Heuristic dùng để làm gì?**
  - Trong những bài toán tìm kiếm trên không gian trạng thái, có 2 trường hợp cần đến heuristic:
    - Vấn đề có thể không có nghiệm chính xác do các mệnh đề không phát biểu chặt chẽ hay thiếu dữ liệu để khẳng định kết quả.
    - Vấn đề có nghiệm chính xác nhưng phí tổn tính toán để tìm ra nghiệm là quá lớn (hệ quả của bùng nổ tổ hợp)
  - **Heuristic giúp tìm kiếm đạt kết quả với chi phí thấp hơn**

Heuristic (tiếp)

- Thuật giải Heuristic là một sự mở rộng khái niệm thuật toán. Nó thể hiện cách giải bài toán với các đặc tính sau:
  - Thường tìm được lời giải tốt (nhưng không chắc là lời giải tốt nhất)
  - Giải bài toán theo thuật giải Heuristic thường dễ dàng và nhanh chóng đưa ra kết quả hơn so với giải thuật tối ưu, vì vậy chi phí thấp hơn.
  - Thuật giải Heuristic thường thể hiện khá tự nhiên, gần gũi với cách suy nghĩ và hành động của con người.

Heuristic (tiếp)

Có nhiều phương pháp để xây dựng một thuật giải Heuristic, trong đó người ta thường dựa vào một số nguyên lý cơ bản như sau:

- Nguyên lý vét cạn thông minh: Trong một bài toán tìm kiếm nào đó, khi không gian tìm kiếm lớn, ta thường tìm cách giới hạn lại không gian tìm kiếm hoặc thực hiện một kiểu dò tìm đặc biệt dựa vào đặc thù của bài toán để nhanh chóng tìm ra mục tiêu.
- Nguyên lý tham lam (Greedy): Lấy tiêu chuẩn tối ưu (trên phạm vi toàn cục) của bài toán để làm tiêu chuẩn chọn lựa hành động cho phạm vi cục bộ của từng bước (hay từng giai đoạn) trong quá trình tìm kiếm lời giải.
- Nguyên lý thứ tự: Thực hiện hành động dựa trên một cấu trúc thứ tự hợp lý của không gian khảo sát nhằm nhanh chóng đạt được một lời giải tốt.
- Hàm Heuristic: Trong việc xây dựng các thuật giải Heuristic, người ta thường dùng các hàm Heuristic. Đó là các hàm đánh giá thô, giá trị của hàm phụ thuộc vào trạng thái hiện tại của bài toán tại mỗi bước giải. Nhờ giá trị này, ta có thể chọn được cách hành động tương đối hợp lý trong từng bước của thuật giải.

Heuristic Greedy

- Bài toán đổi tiền: Đổi số tiền n thành các loại tiền cho trước sao cho số tờ là ít nhất
- Bài toán hành trình ngắn nhất (TSP): Hãy tìm một hành trình cho một người giao hàng đi qua n điểm khác nhau, mỗi điểm đi qua một lần và trở về điểm xuất phát sao cho tổng chiều dài đoạn đường cần đi là ngắn nhất. Giả sử rằng có con đường nối trực tiếp từ giữa hai điểm bất kỳ.
  - Vét cạn: (n-1)! (Với n lớn ???)
  - Greedy 1: Mỗi bước chọn i →j sao cho j gần i nhất trong những đỉnh nối với i còn lại
  - Greedy 2: Mỗi bước chọn i →j sao cho i gần j nhất trong những đỉnh nối với j còn lại

Ví dụ: TSP với n=8

	1	2	3	4	5	6	7	8
1	0	730	640	840	800	430	380	1010
2	730	0	710	1040	500	300	540	470
3	640	710	0	1420	1050	600	920	1160
4	840	1040	1420	0	740	950	570	900
5	800	500	1050	740	0	520	460	200
6	430	300	600	950	520	0	390	690
7	380	540	920	570	460	390	0	660
8	1010	470	1160	900	200	690	660	0

Ví dụ: TSP với n=8

*Với Greedy 1:

$1 \rightarrow 7 \rightarrow 6 \rightarrow 2 \rightarrow 8 \rightarrow 5 \rightarrow 4 \rightarrow 3 \rightarrow 1$

Tổng chi phí: 4540

*Với Greedy 2:

$1 \rightarrow 7 \rightarrow 4 \rightarrow 5 \rightarrow 8 \rightarrow 2 \rightarrow 6 \rightarrow 3 \rightarrow 1$

Tổng chi phí: 3900

Bài toán 3: Bài toán tô màu bản đồ

Heuristic (tt)

- Heuristic dùng như thế nào trong tìm kiếm?
  - Tìm kiếm trên không gian trạng thái theo chiều nào? Sâu hay rộng?
  - Tìm theo Heuristic : Heuristic định hướng quá trình tìm kiếm theo hướng mà "nó" cho rằng khả năng đạt tới nghiệm là cao nhất. Không "sâu" cũng không "rộng"
- Kết quả của tìm kiếm với Heuristic
  - Việc tìm kiếm theo định hướng của heuristic có kết quả tốt hay xấu tùy theo heuristic "đúng" hay "sai".
  - Heuristic có khả năng bỏ sót nghiệm
  - Heuristic càng tốt càng dẫn đến kết quả nhanh và tốt.
  - **Làm sao tìm được Heuristic tốt???**

3.2 Tìm kiếm tối ưu (Best First Search)

OPEN : tập chứa các trạng thái đã được sinh ra nhưng chưa được xét đến (vì ta đã chọn một trạng thái khác). Thực ra, OPEN là một loại hàng đợi ưu tiên (priority queue) mà trong đó, phần tử có độ ưu tiên cao nhất là phần tử *tốt nhất*.
CLOSE : tập chứa các trạng thái đã được xét đến. Chúng ta cần lưu trữ những trạng thái này trong bộ nhớ để đề phòng trường hợp khi một trạng thái mới được tạo ra lại trùng với một trạng thái mà ta đã xét đến trước đó.

Thuật giải BEST-FIRST SEARCH

1. Đặt OPEN chứa trạng thái khởi đầu.
2. Cho đến khi tìm được trạng thái đích hoặc không còn nút nào trong OPEN, thực hiện :
    2.a. Chọn trạng thái tốt nhất (Tmax) trong OPEN (và xóa Tmax khỏi OPEN)
    2.b. Nếu Tmax là trạng thái kết thúc thì thoát.
    2.c. Ngược lại, tạo ra các trạng thái kế tiếp T_k có thể có từ trạng thái Tmax.
    Đối với mỗi trạng thái kế tiếp T_k thực hiện : Tính $f(T_k)$; Thêm T_k vào OPEN

3.2 Tìm kiếm tối ưu (tiếp)

Thuật giải BEST-FIRST SEARCH

```
Begin
    open:={s};
    While (open<> ø) do
        begin
            n:= Retrieve(Open) //Chọn trạng thái tốt nhất từ Open.
            if (n=g) then return True
            else begin
                Tạo Γ(n)
                 for mỗi nút con m của Γ(n) do
                        Gán giá trị chi phí cho m
                                Open:=Open∪{m};
        end;
    Return False;
End;
```

```
Begin
        Open := {s};
        Close := ø;
        While (Open <> ø) do
                begin
                        n:=Retrieve(Open);
                        if (n=g) then Return True;
                        Open := Open ∪ Γ(n); // (Γ(n) – Close)
                        Close := Close ∪ {n};
                end;
        Return False;
End;
```

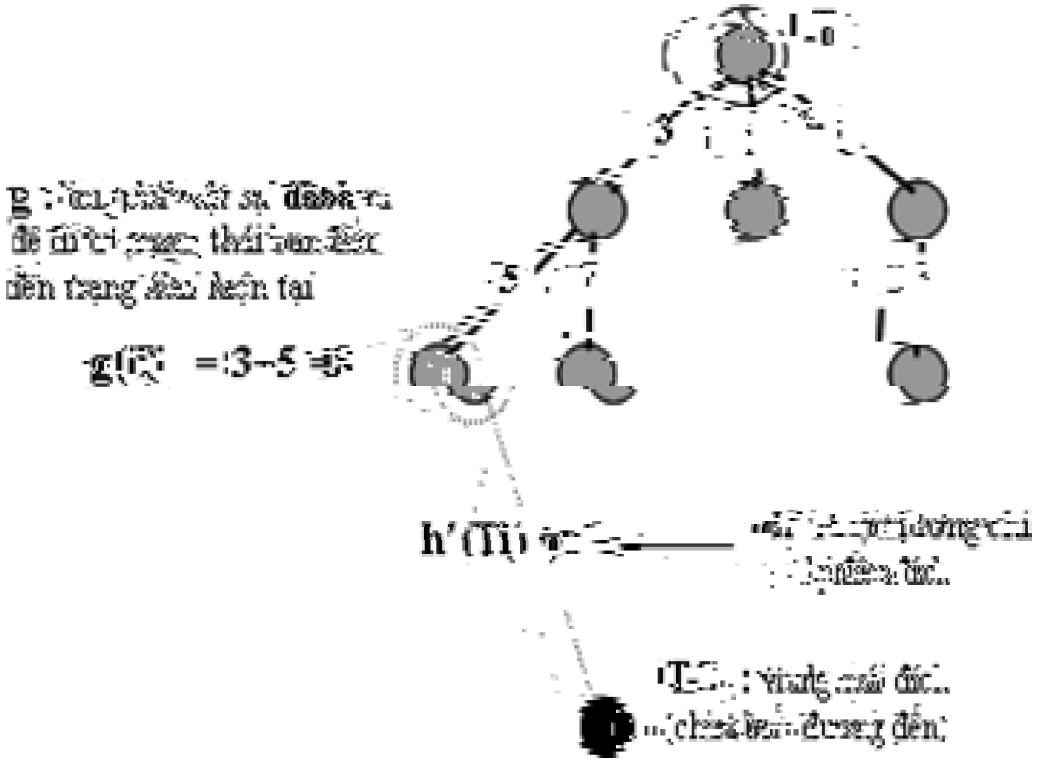
3.2 Tìm kiếm tối ưu (tiếp)

- BFS khá đơn giản. Tuy vậy, trên thực tế, cũng như tìm kiếm chiều sâu và chiều rộng, hiếm khi ta dùng BFS một cách trực tiếp. Thông thường, người ta thường dùng các phiên bản của BFS là AT, AKT và A*

Thông tin về quá khứ và tương lai

- Thông thường, trong các phương án tìm kiếm theo kiểu BFS, độ tốt **f** của một trạng thái được tính dựa theo 2 hai giá trị mà ta gọi là là **g** và **h'**. **h'** chúng ta đã biết, đó là một ước lượng về chi phí từ trạng thái hiện hành cho đến trạng thái đích (thông tin tương lai). Còn **g** là "chiều dài quãng đường" *đã* đi từ trạng thái ban đầu cho đến trạng thái hiện tại (thông tin quá khứ). Lưu ý rằng **g** là chi phí thực sự (không phải chi phí ước lượng).

3.3 Thuật giải A^T



Phân biệt khái niệm g và h'

3.3 Thuật giải A^T

Thuật giải A^T là một phương pháp tìm kiếm theo kiểu BFS với độ tốt của nút là giá trị hàm g – tổng chiều dài con đường đã đi từ trạng thái bắt đầu đến trạng thái hiện tại.

```
Begin
   open:={s};
   While (open<> ø) do
      begin
         n:= Retrieve(Open)      //Chọn n sao cho g(n) →nhỏ nhất từ Open.
         if (n=g) then return True
         else begin
            Tạo Γ(n)
             for mỗi nút con m của Γ(n) do
                     if (m∉ Open) then
                       Begin
                          g(m):=g(n)+Cost(n,m)
                        Open:=Open∪{m};
                       end
                     else So sánh g(m) va g_New (m) và cập nhật
      end;
   Return False;
```

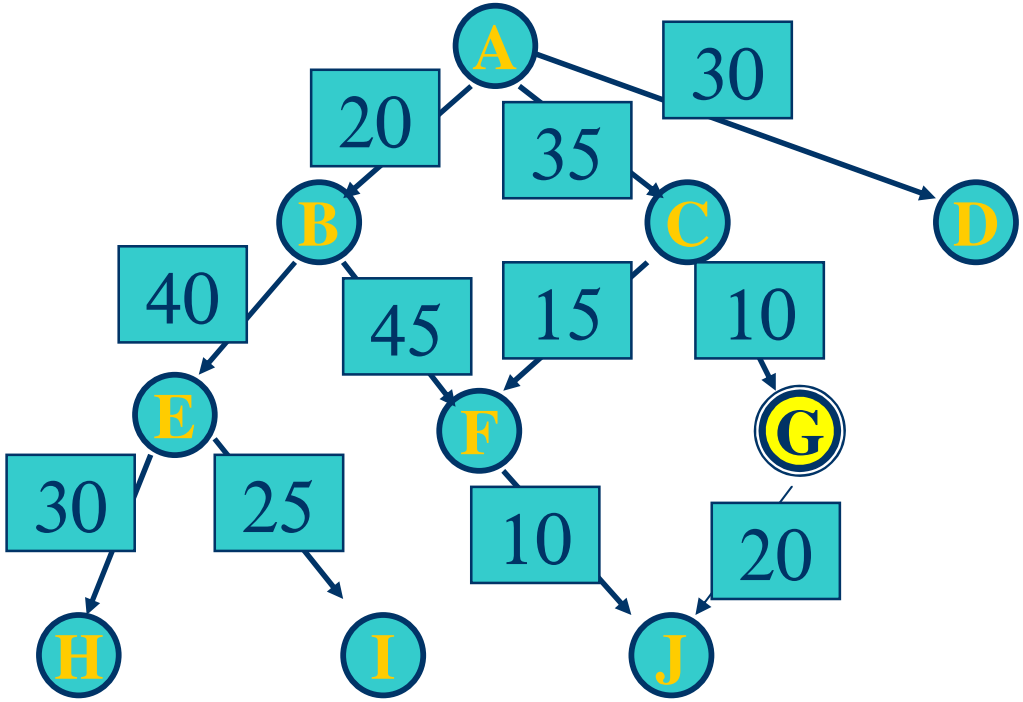
3.3 Thuật giải CMS (Cost Minimazation Search)

Thuật giải CMS là một phương pháp tìm kiếm theo kiểu BFS với độ tốt của nút là giá trị hàm g và bổ sung tập Close: tập đỉnh đã xét).

Begin
open:={s}; close := ø
While (open<> ø) **do**
begin
n:= Retrieve(Open) //Chọn n sao cho g(n) →nhỏ nhất từ Open.
if (n=g) **then** return True
else begin
Tạo Γ(n)
for mỗi nút con m của Γ(n) **do**
if (m∉ Open) and (m∉ Close) then
Begin
g(m):=g(n)+Cost(n,m)
Open:=Open∪{m};
end
else So sánh g(m) va g_{New} (m) và cập nhật
close = close ∪{n}
end;
Return False;
End;

Ví dụ:

- Xét graph sau:



s = A là đỉnh bắt đầu

g= J là đỉnh đích

Ví dụ:

- :

Trước	Sau
*	A
A	B
A	C
A	D
B	E
C	F
C	G
F	J

s = A là đỉnh bắt đầu

g= J là đỉnh đích

Lần lặp	n	Γ(n)	Open
0			{(A,0)}
1	A	{B,C,D}	{(B,20), (C,35), (D,30)}
	B	{E,F}	(C,35), (D,30),(E,60),(F,65)
	D	ø	(C,35),(E,60),(F,65)
	C	{F,G}	(E,60),(F,50),(G,45)
	G	{J}	(E,60),(F,50),(J,65)
	F	(J}	(E,60),(J,60)
	J		

3.4 Thuật giải A^{KT}

(Algorithm for Knowlegeable Tree Search)

Thuật giải A^{KT} mở rộng A^{T} bằng cách sử dụng thêm thông tin ước lượng h'. Độ tốt của một trạng thái f là tổng của hai hàm g và h'.

```
Begin
    open:={s};
    While (open<> ø) do
        begin
            n:= Retrieve(Open)      //Chọn n sao cho f(n) →nhỏ nhất từ Open.
            if (n=g) then return True
            else begin
                Tạo Γ(n)
                 for mỗi nút con m của Γ(n) do
                        Begin
                            g(m):=g(n)+Cost(n,m)
                            f(m):= g(m)+h'(m);
                           Open:=Open∪{m};
                        end;
        end;
    Return False;
End;
```

Thuật giải A*

A* là một phiên bản đặc biệt của A^{KT} áp dụng cho trường hợp đồ thị. Thuật giải A* có sử dụng thêm tập hợp CLOSE để lưu trữ những trường hợp đã được xét đến. A* mở rộng A^{KT} bằng cách bổ sung cách giải quyết trường hợp khi "mở" một nút mà nút này đã có sẵn trong OPEN hoặc CLOSE.

3.5 Thuật giải A* (tiếp)

```
Begin
 open:={s}; close:=ø;
While (open<> ø) do
 begin
    n:= Retrieve(Open) //sao cho f(n) min.
    if (n=g) then return path từ s đến g
    else begin
       Tạo Γ(n)
        for mỗi nút con m của Γ(n) do
          case m of
              m ∉ Open và m ∉ Close:
                 begin
                     Gán giá trị heuristic cho m
                     Open:=Open∪{m};
                 end;
              m ∈ Open:
                 if đến được m bằng một path ngắn hơn
                 then Cập nhật lại m trong Open.
              m ∈ Close
                 if đến được m bằng một path ngắn hơn
                 then begin
                              Close:=Close-{m}
                              Open:=Open∪{m}
                      end;
          end; /*end case*/
      Close:=Close∪{n}
     end; / while/
     return false;
End;
```

Hàm lượng giá Heuristic

- Hàm lượng giá Heuristic là hàm ước lượng phí tổn để đi từ trạng thái hiện tại đến trạng thái goal.
- Cơ sở để xác định hàm lượng giá là dựa vào tri thức/kinh nghiệm thu thập được.
- Hàm lượng giá cho kết quả đúng (gần thực thế) hay sai (xa giá trị thực) sẽ dẫn đến kết quả tìm được tốt hay xấu.
- Không có chuẩn mực cho việc đánh giá một hàm lượng giá Heuristic. Lý do:
  - Không có cấu trúc chung cho hàm lượng giá
  - Tính đúng/sai thay đổi liên tục theo từng vấn đề cụ thể
  - Tính đúng/sai thay đổi theo từng tình huống cụ thể trong một vấn đề
- **Có thể dùng nhiều hàm lượng giá khác nhau theo tình huống ➔ cần hàm lượng giá về các hàm lượng giá.**

Trò đố 8 ô hay 15 ô

Trạng thái ban đầu Trạng thái đích

- Trò đố 15 ô

11	14	4	7
10	6		5
1	2	13	15
9	12	8	3

1	2	3	4
12	13	14	5
11		15	6
10	9	8	7

- Trò đố 8 ô

	2	8
3	5	7
6	2	1

1	2	3
8		4
7	6	5

- Cần biểu diễn KGTT cho bài toán này như thế nào?

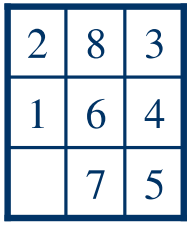
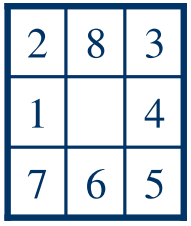
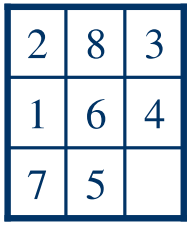
Thuật giải A* – Ví dụ

- Xét bài toán 8 pussle với goal là:

1	2	3
8		4
7	6	5

Heuristic 1: Tổng số miếng sai vị trí

Heuristic 2: Tổng khoảng cách sai vị trí của từng miếng.

	5	6	
	3	4	
	5	6	

Việc chọn lựa hàm Heuristic là khó khăn và có ý nghĩa quyết định đối với tốc độ của giải thuật

Hàm lượng giá Heuristic – Cấu trúc

- Xét lại hoạt động của giải thuật Best First Search:
  - Khi có 2 nút cùng có giá trị kỳ vọng đạt đến mục tiêu bằng nhau thì nút có path từ nút bắt đầu đến nút đó ngắn hơn sẽ được chọn trước như vậy nút này có giá trị Heuristic tốt hơn.
  - Hay nói cách khác hàm lượng giá Heuristic cho nút gần start hơn là tốt hơn nếu kỳ vọng đến goal là bằng nhau.
  - Vậy chọn nút nào nếu kỳ vọng của 2 nút khác nhau? Nút kỳ vọng tốt hơn nhưng xa start hay nút kỳ vọng xấu hơn nhưng gần root

Hàm lượng giá bao gồm cả 2 và có cấu trúc:

F(n) := G(n) + H(n)

G(n): phí tổn thực từ root đến n

H(n): phí tổn ước lượng heuristic từ n đến goal.

Thuật giải A* – Ví dụ

- Xét ví dụ là bài toán 8 puzzle với:

2	8	3
1	6	4
7		5

Bắt đầu

1	2	3
8		4
7	6	5

Đích

Hàm lượng giá: F(n) = G(n) + H(n)

Với G(n): số lần chuyển vị trí đã thực hiện

H(n): Số miếng nằm sai vị trí

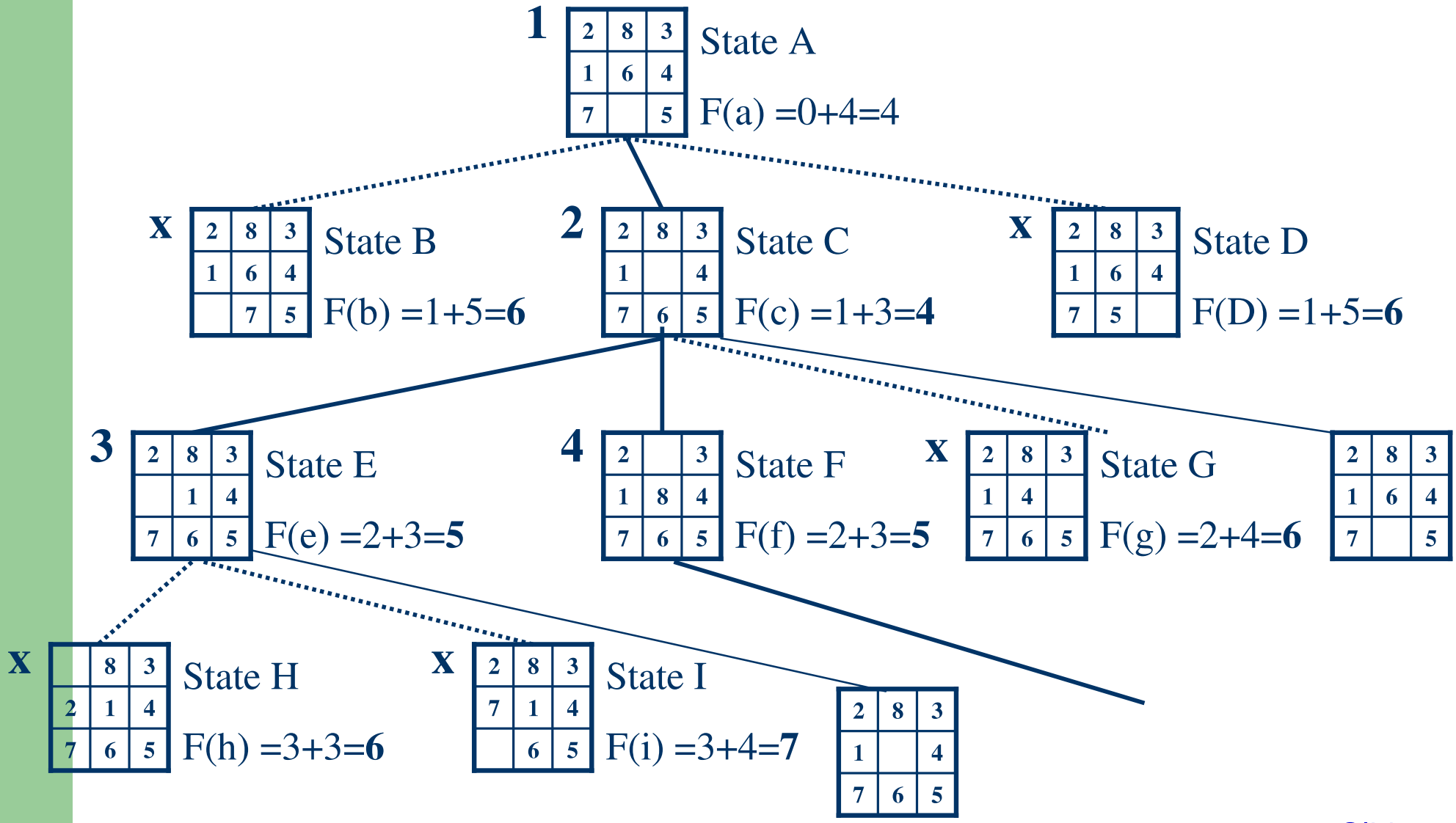
Nút X có giá trị heuristic tốt hơn nút Y nếu **F(x) < F(y)**.

Ta có hoạt động của giải thuật Best First search trên như hình sau:

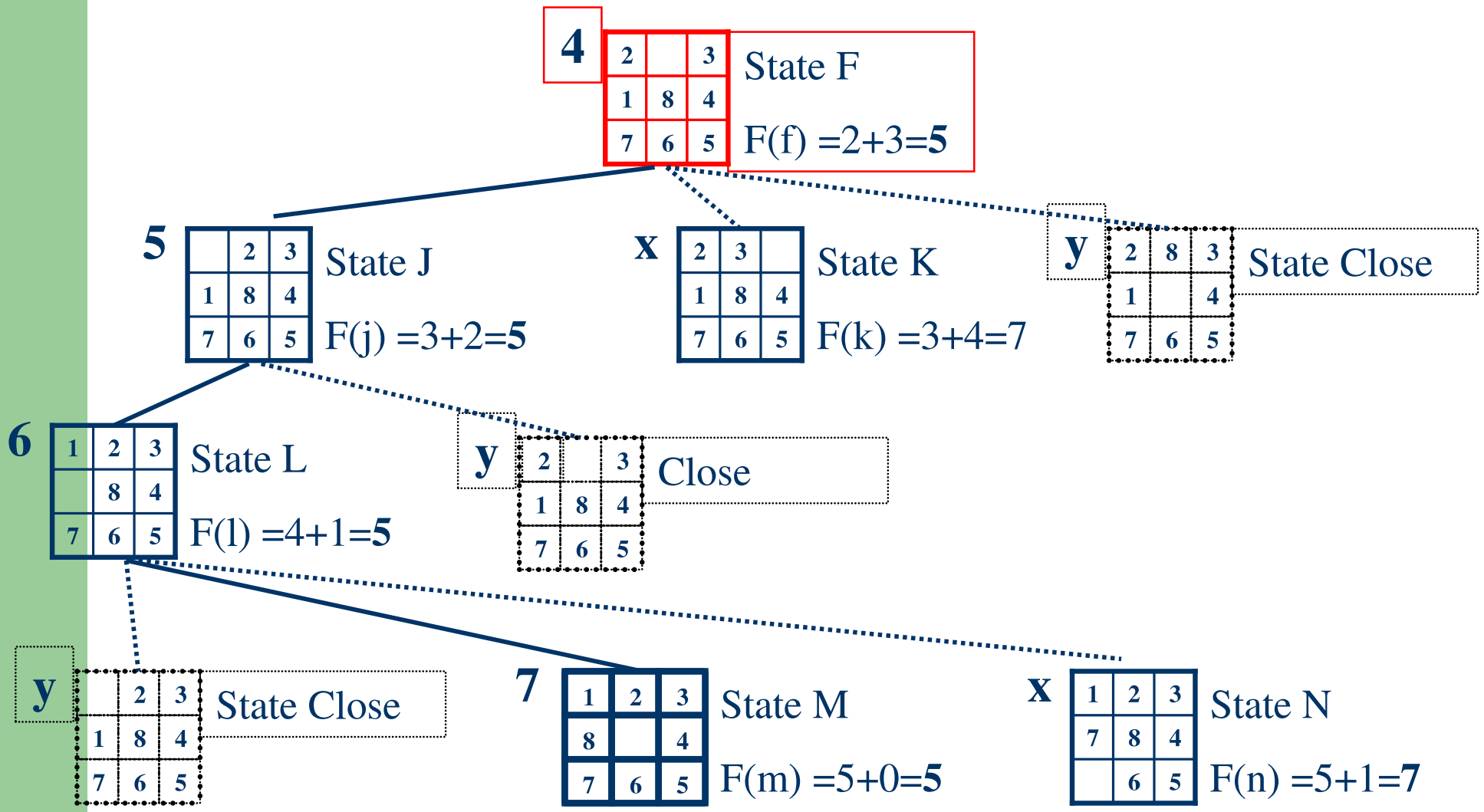
3.5 Thuật giải A* (tiếp)

```
Begin
 open:={s}; close:=ø;
While (open<> ø) do
 begin
    n:= Retrieve(Open) //sao cho f(n) min.
    if (n=g) then return path từ s đến g
    else begin
       Tạo Γ(n)
        for mỗi nút con m của Γ(n) do
          case m of
              m ∉ Open và m ∉ Close:
                 begin
                     Gán giá trị heuristic cho m
                     Open:=Open∪{m};
                 end;
              m ∈ Open:
                 if đến được m bằng một path ngắn hơn
                 then Cập nhật lại m trong Open.
              m ∈ Close
                 if đến được m bằng một path ngắn hơn
                 then begin
                              Close:=Close-{m}
                              Open:=Open∪{m}
                      end;
          end; /*end case*/
      Close:=Close∪{n}
     end; / while/
     return false;
End;
```

Ví dụ

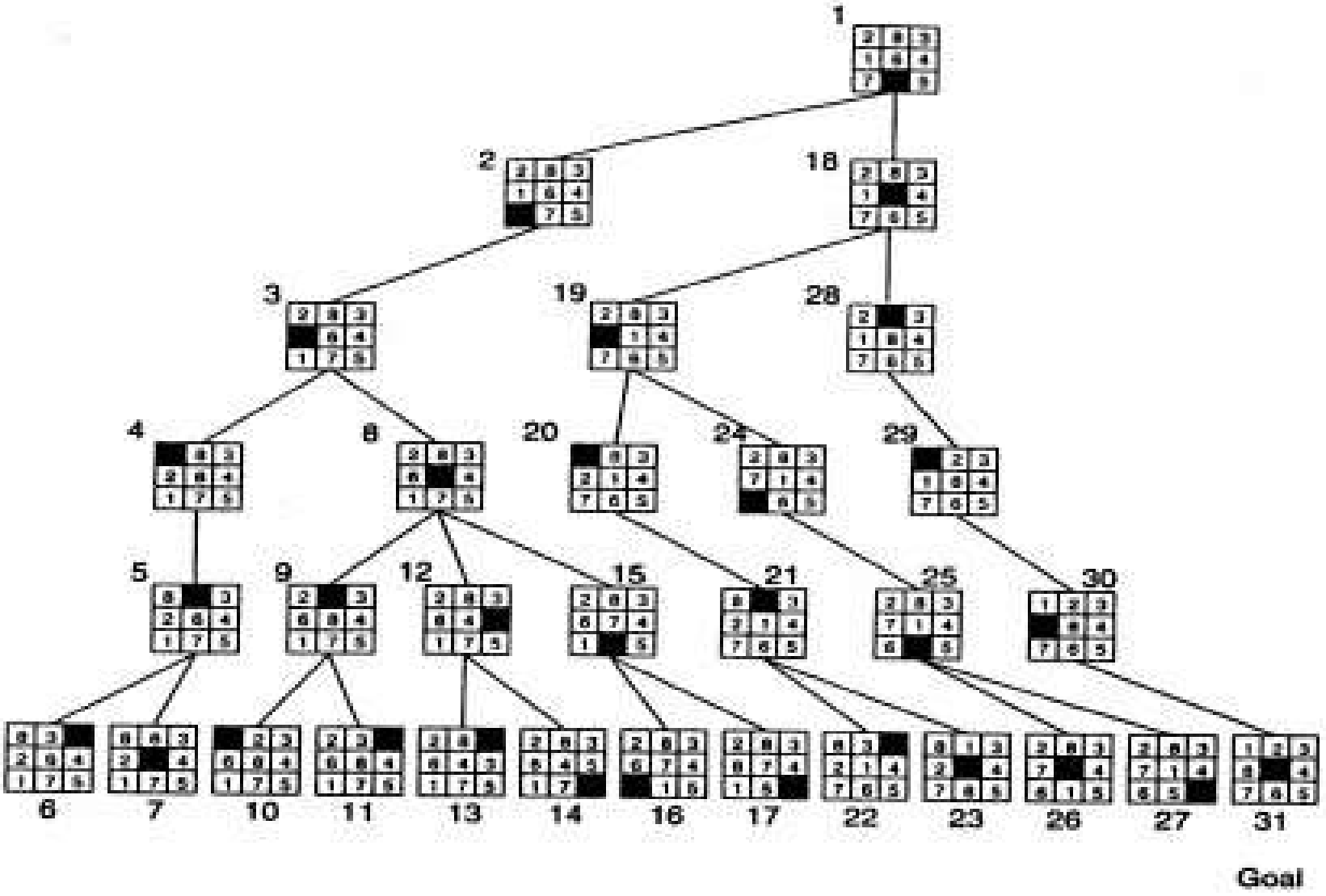

1
2 8 3
1 6 4
7 5
State A
F(a) =0+4=4
x
2 8 3
1 6 4
7 5
State B
F(b) =1+5=6
2
2 8 3
1 4
7 6 5
State C
F(c) =1+3=4
x
2 8 3
1 6 4
7 5
State D
F(D) =1+5=6
3
2 8 3
1 4
7 6 5
State E
F(e) =2+3=5
4
2 3
1 8 4
7 6 5
State F
F(f) =2+3=5
x
2 8 3
1 4
7 6 5
State G
F(g) =2+4=6
2 8 3
1 6 4
7 5
x
8 3
2 1 4
7 6 5
State H
F(h) =3+3=6
x
2 8 3
7 1 4
6 5
State I
F(i) =3+4=7
2 8 3
1 4
7 6 5

Ví dụ


4
2 3
1 8 4
7 6 5
State F
F(f) =2+3=5
5
2 3
1 8 4
7 6 5
State J
F(j) =3+2=5
x
2 3
1 8 4
7 6 5
State K
F(k) =3+4=7
y
2 8 3
1 4
7 6 5
State Close
6
1 2 3
8 4
7 6 5
State L
F(l) =4+1=5
y
2 3
1 8 4
7 6 5
Close
y
2 3
1 8 4
7 6 5
State Close
7
1 2 3
8 4
7 6 5
State M
F(m) =5+0=5
x
1 2 3
7 8 4
6 5
State N
F(n) =5+1=7

Trò chơi ô đố 8-puzzle

The 8-puzzle searched by a production system with loop detection and depth bound 5



Hoạt động theo giải thuật A*

Lần	n	Open	Close
0		{A4}	{}
1	A4	{C4,B6,D6}	{A4}
2	C4	{E5,F5,G6,B6,D6}	{A4,C4}
3	E5	{F5,H6,G6,B6,D6,I7}	{A4,C4,E5}
4	F5	{J5,H6,G6,B6,D6,K7,I7}	{A4,C4,E5,F5}
5	J5	{L5,H6,G6,B6,D6,K7,I7}	{A4,C4,E5,F5,J5}
6	l5	{M5,H6,G6,B6,D6,K7,I7,N7}	{A4,C4,E5,F5,J5,L5}
7	**m5**		

Đánh giá giải thuật Heuristic

- **Admissibility** – Tính chấp nhận
  - Một giải thuật Best first search với hàm đánh giá
  - F(n) = G(n) + H(n) với
    - N : Trạng thái bất kỳ
    - G(n) : Phí tổn đi từ nút bắt đầu đến nút n
    - H(n) : Phí tổn ước lượng heuristic đi từ nút **n** đến **goal**
  - Được gọi là giải thuật **A**
- Một giải thuật tìm kiếm được xem là admissible nếu đối với một đồ thị bất kỳ nó luôn dừng ở path nghiệm tốt nhất (nếu có).
- **Giải thuật A***: Là giải thuật A với hàm heuristic H(n)luôn luôn ≤ giá trị thực đi từ n đến goal.
- **Giải thuật A* là admissible**

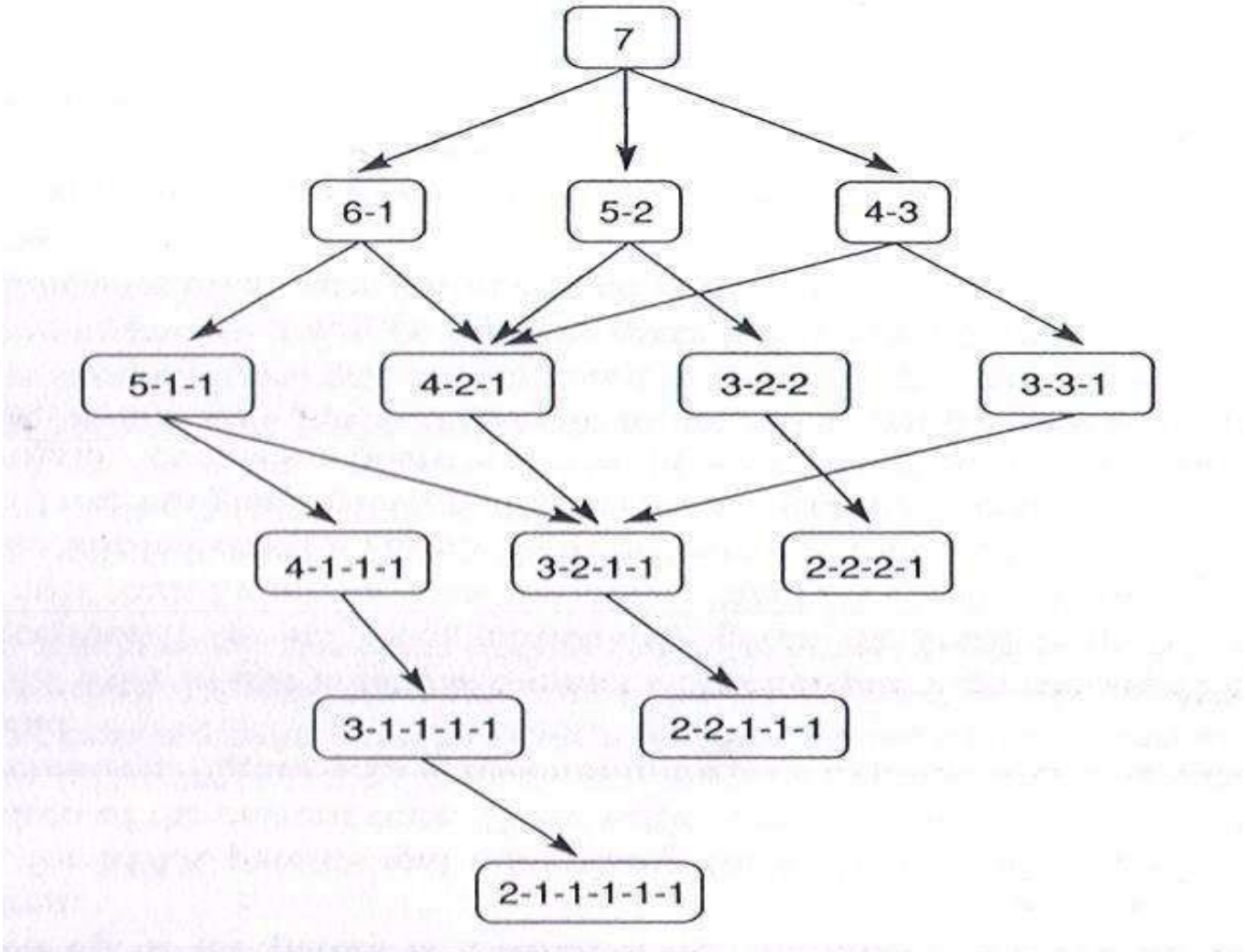
Đánh giá giải thuật Heuristic

- **Monotonicity** – Đơn điệu
  - Một hàm heuristic H(n) được gọi là monotone (đơn điệu) nếu:
  - $\forall n_i, n_j$: n_j là nút con cháu của ni ta có
    $H(n_i)$-$H(n_j) \leq$ phí tổn thật đi từ ni đến nj
  - Đánh giá heuristic của đích là 0 : H(goal) = 0.
- Giải thuật A có hàm H(n) monotone là giải thuật A* và Admissible
- **Informedness**
- Xét 2 hàm heuristic $H_1(n)$ và $H_2(n)$ nếu ta có $H_1(n) \leq H_2(n)$ với mọi trạng thái n thì $H_2(n)$ được cho là informed hơn $H_1(n)$.

Heuristic trong trò chơi đối kháng

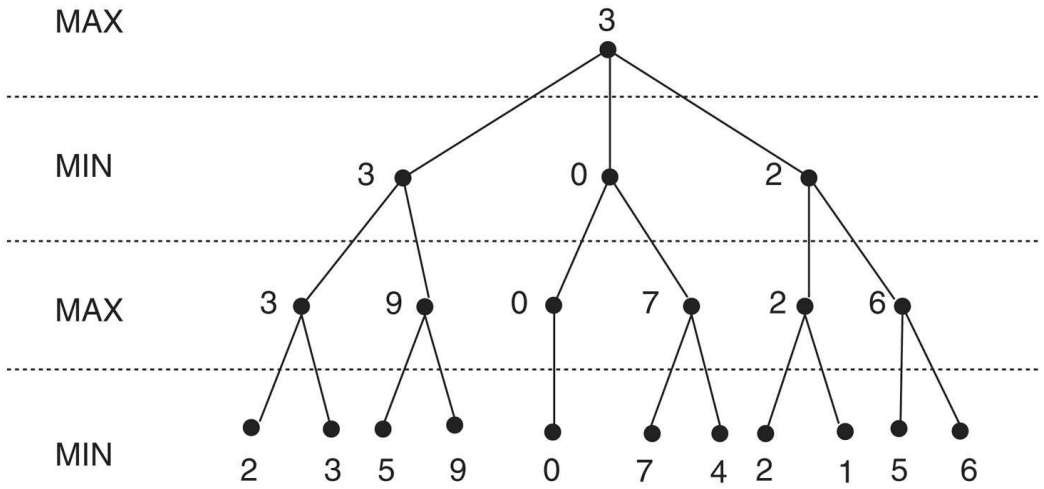
- **Giải thuật minimax**:
  - Hai đấu thủ trong trò chơi được gọi là **MIN** và **MAX**.
  - Mỗi nút lá có giá trị:
    - **1** nếu là MAX thắng,
    - **0** nếu là MIN thắng.
  - Minimax sẽ truyền các giá trị này lên cao dần trên đồ thị, qua các nút cha mẹ kế tiếp theo các luật sau:
    - Nếu trạng thái cha mẹ là **MAX**, gán cho nó giá trị **lớn nhất** có trong các trạng thái con.
    - Nếu trạng thái bố, mẹ là **MIN**, gán cho nó giá trị **nhỏ nhất** có trong các trạng thái con.

Hãy áp dụng GT Minimax vào Trò Chơi NIM



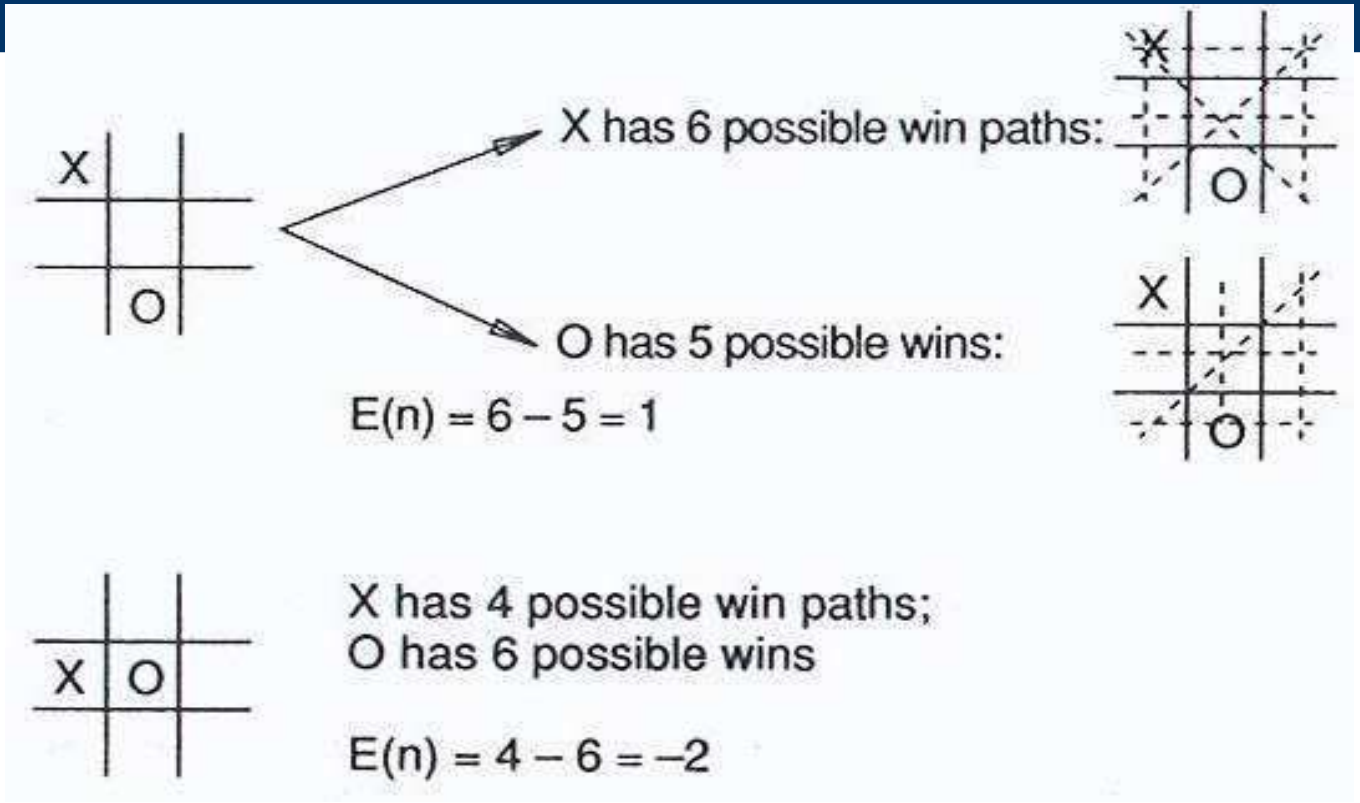
Minimax với độ sâu lớp cố định

- Minimax đối với một KGTT giả định.



- Các nút lá được gán các giá trị ***heuristic***
- Còn giá trị tại các nút trong là các giá trị nhận được dựa trên giải thuật Minimax

Heuristic trong trò chơi tic-tac-toe



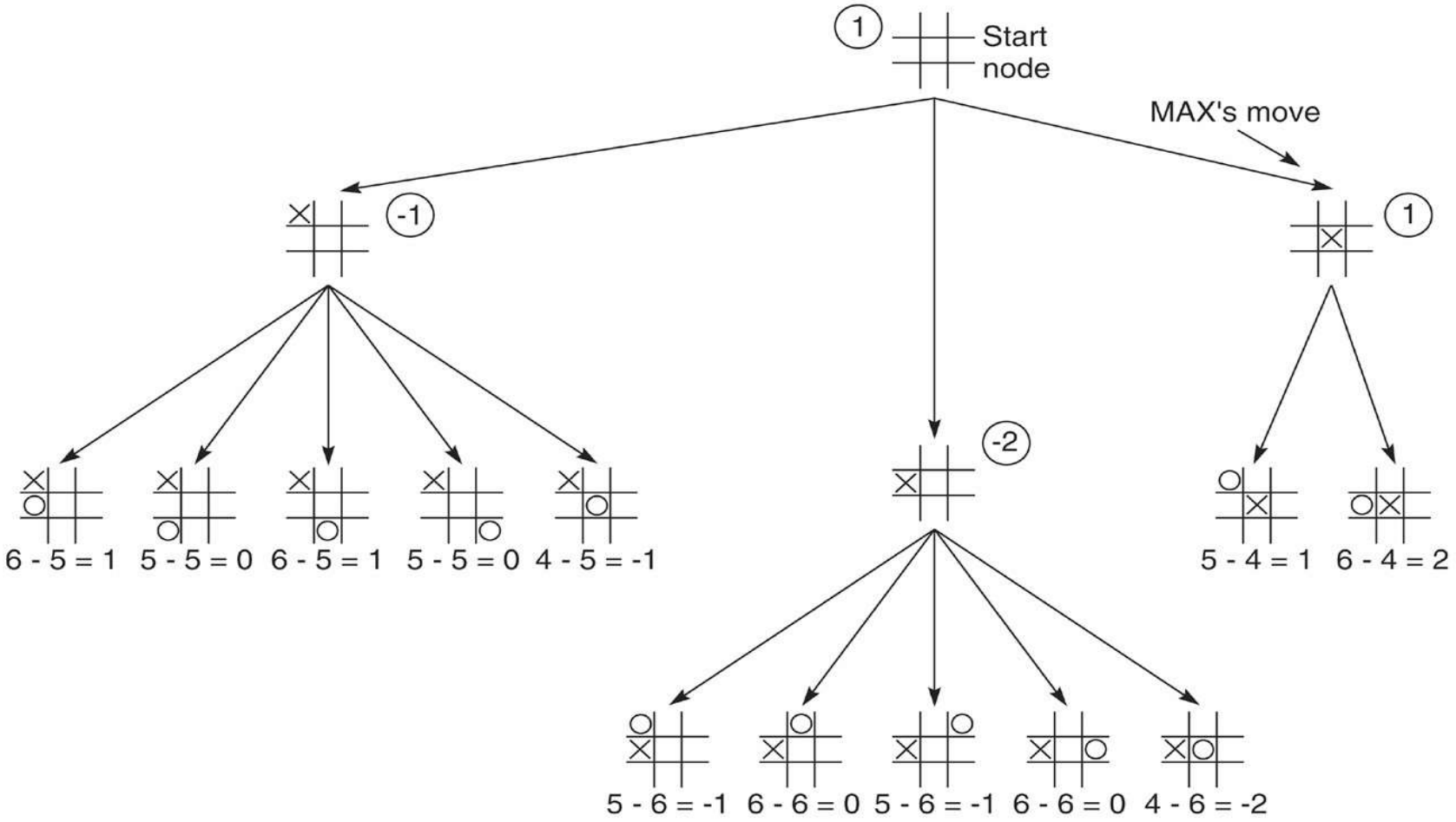
Hàm Heuristic: **E(n) = M(n) – O(n)**

Trong đó: M(n) là *tổng số đường thắng có thể của tôi*

O(n) là *tổng số đường thắng có thể của đối thủ*

E(n) là *trị số đánh giá tổng cộng cho trạng thái n*

Minimax 2 lớp trong tic-tac-toe

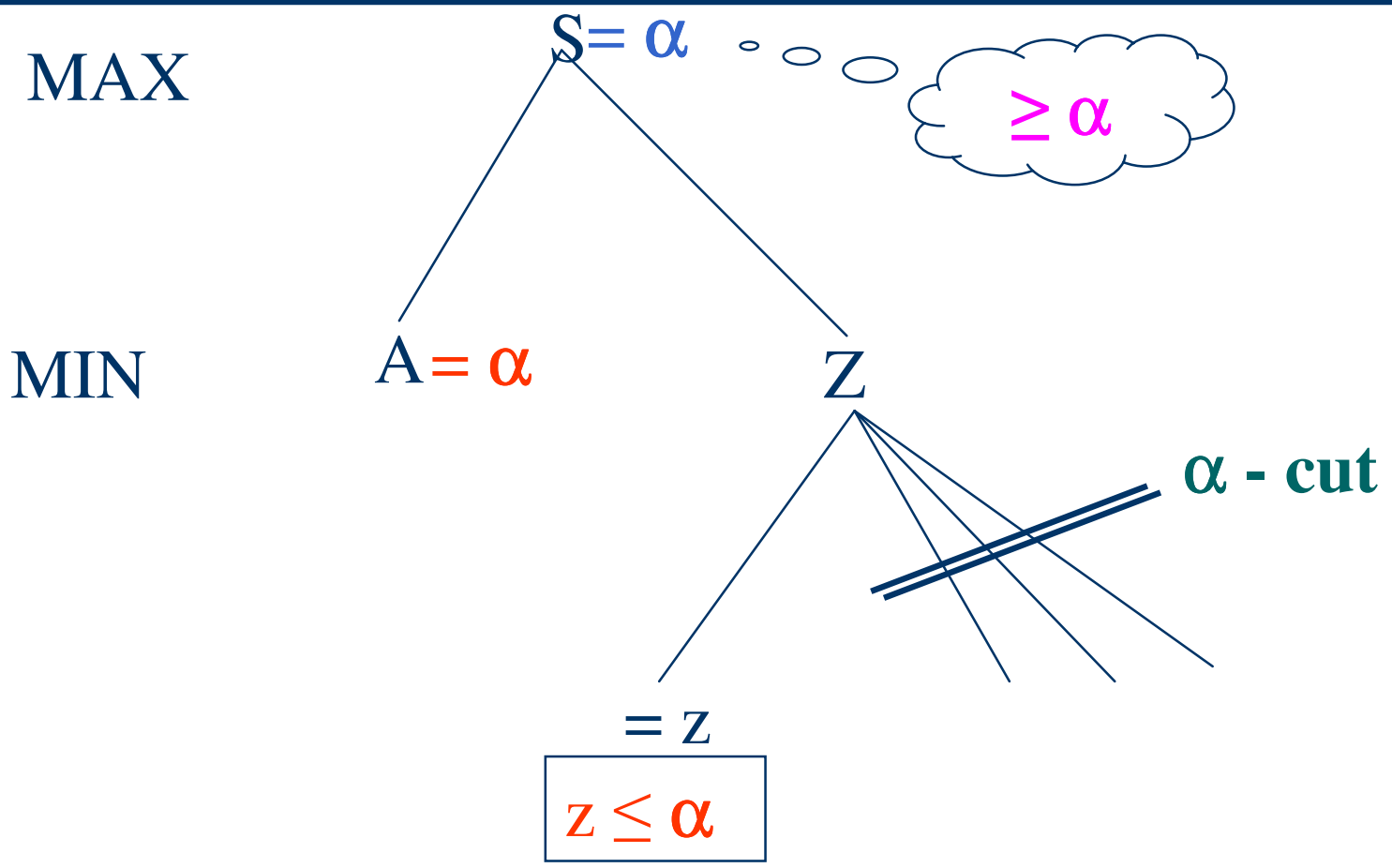


Trích từ Nilsson (1971).

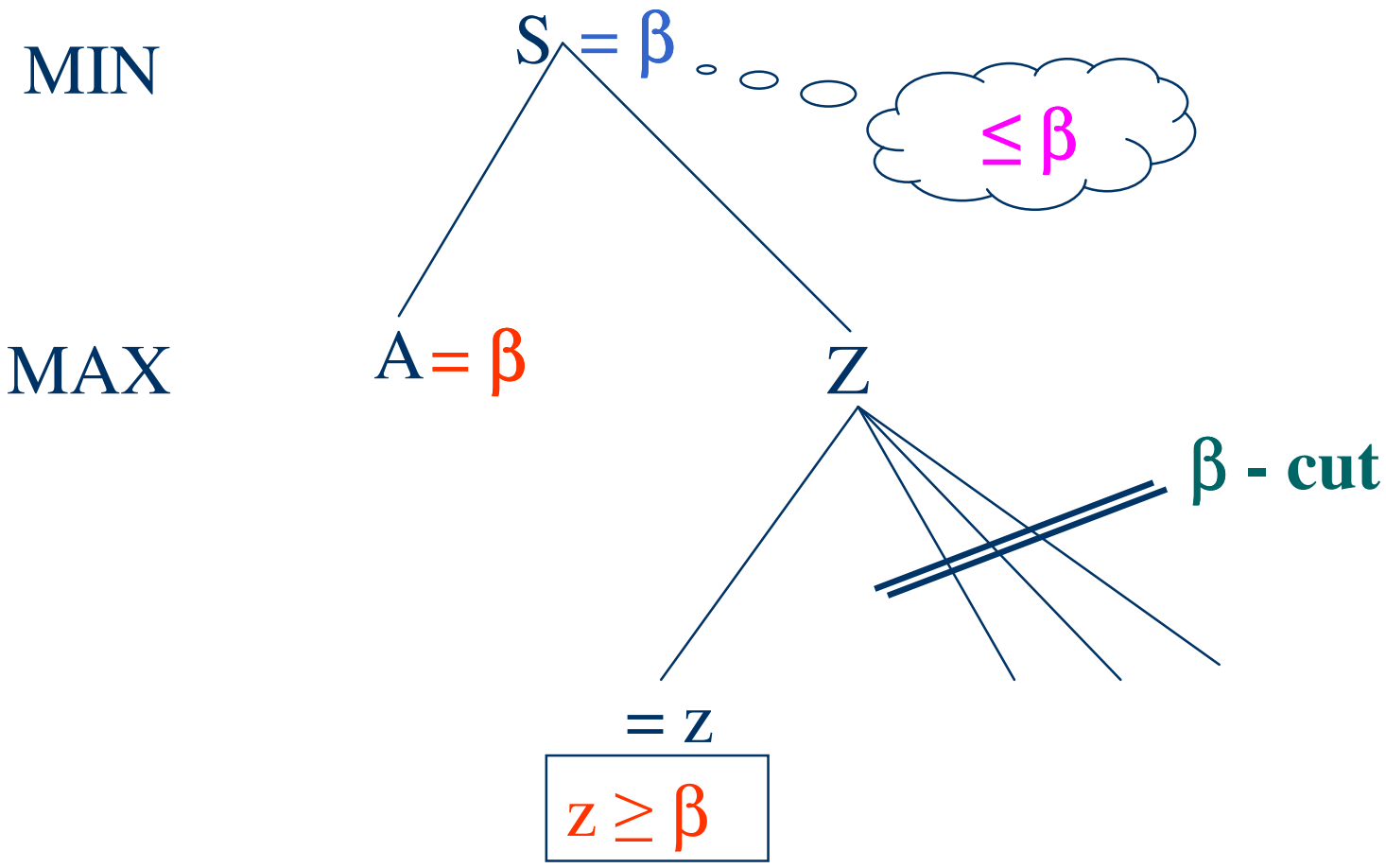
Giải thuật cắt tỉa α-β

- Tìm kiếm theo kiểu depth-first.
- Nút MAX có 1 giá trị α (luôn tăng)
- Nút MIN có 1 giá trị β (luôn giảm)
- **TK có thể kết thúc dưới bất kỳ**:
  - Nút MIN nào có $\beta \leq \alpha$ của bất kỳ nút cha MAX nào.
  - Nút MAX nào có $\alpha \geq \beta$ của bất kỳ nút cha MIN nào.
- Giải thuật cắt tỉa α-β thể hiện *mối quan hệ giữa các nút ở lớp n và n+2*, mà tại đó toàn bộ cây có gốc tại lớp n+1 có thể cắt bỏ.

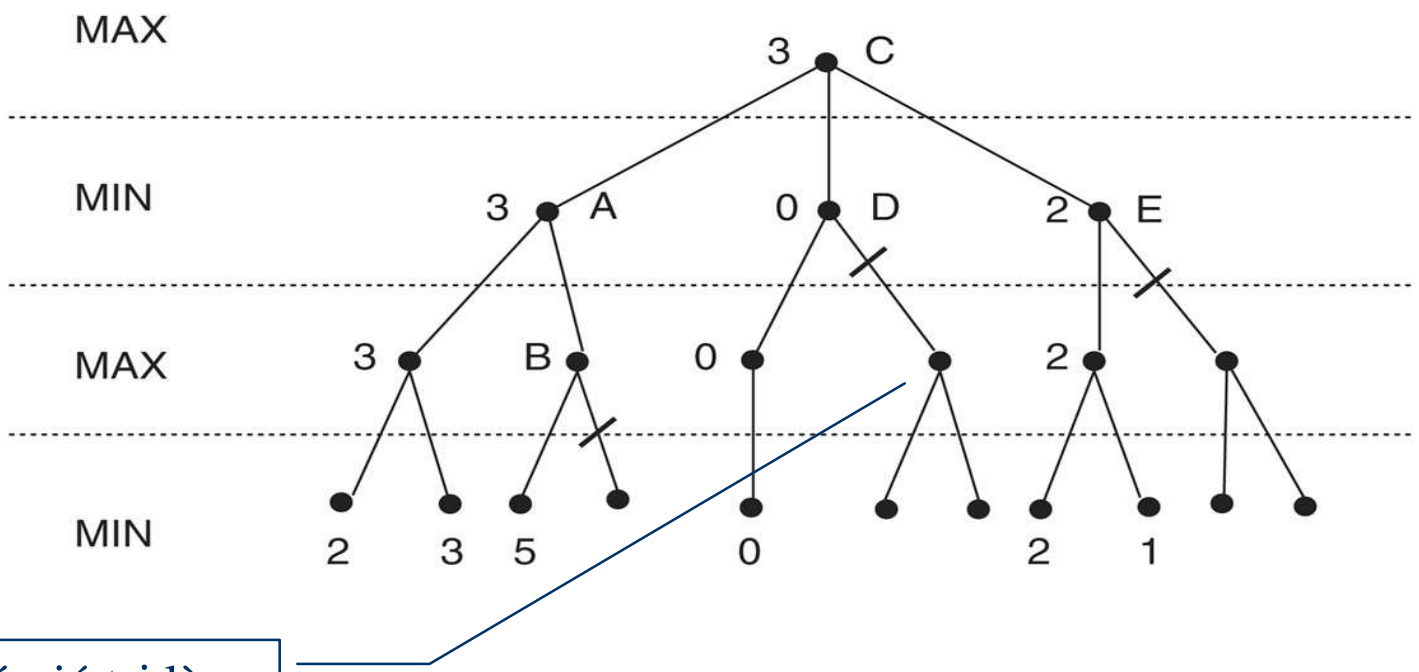
Cắt tỉa α



Cắt tỉa β



GT Cắt Tỉa α-β áp dụng cho KGTT giả định



Các nút không có giá trị là các nút không được duyệt qua

A has $\beta = 3$ (A will be no larger than 3)
B is β pruned, since 5 > 3
C has $\alpha = 3$ (C will be no smaller than 3)
D is α pruned, since 0 < 3
E is α pruned, since 2 < 3
C is 3